

Unit 4

Relational Data Model

Dr. Dhanraj Verma
Professor, CSE-AIML





Topic

Relational Data Model



What is Relational Data Model?

The relational model is defined as a collection of tables to represent both data and the relationships among those data.

Tables are known as *Relations*. Each table has multiple rows and columns, and each column has a unique name. Columns are known as *Attributes*. Rows are known as *Tuples*.

In this *Employee* relation, there are four attributes named *ID*, *Name*, *Department* and *Salary*.

ID	Name	Department	Salary
28283	A	CSE	30000
32442	B	EE	35000
43553	C	MT	53000



Degree

The degree is the number of entities involved in a particular relationship. It refers to the number of different attributes or columns in a table.

- The degree of relationship can be categorized as Unary, Binary, Ternary & N-ary relationship.
- In a unary relationship, only one entity is involved. Here, the degree of relationship is 1.
- In a binary relationship, there are two entities involved. The degree of relationship is 2.
- In a ternary relationship, there are three entities involved. The degree of relationship is 3.
- In an N-ary relationship, there is an n number of involved entities. The degree of relationship is 'n'.

Cardinality

The uniqueness of data present in a column is called “Cardinality”.

A column with a high cardinality means that a significant percentage of the values are unique.

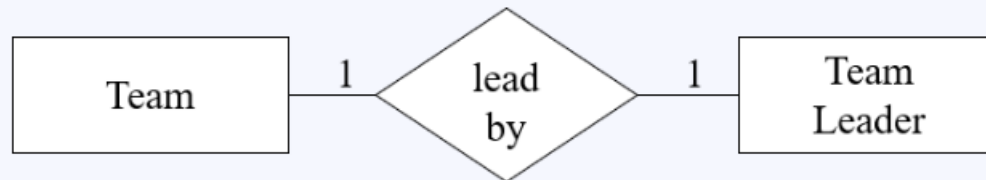
Low cardinality refers to a column with a large number of repeated values.

A greater cardinality indicates more distinct values, leading to improved selectivity and faster query execution.

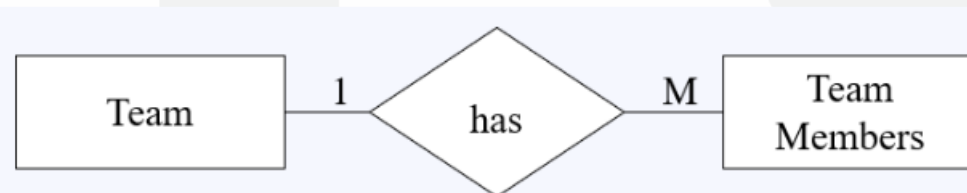
A reduced cardinality, may suggest a limited number of distinct values, allowing the database to use specialized optimization techniques.

Types of Cardinality

- One-to-One cardinality (1:1)

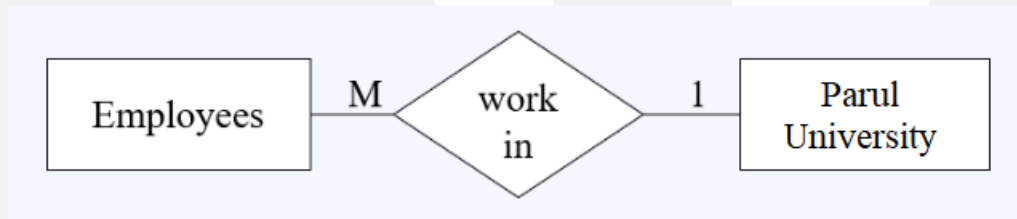


- One-to-Many cardinality (1:m)

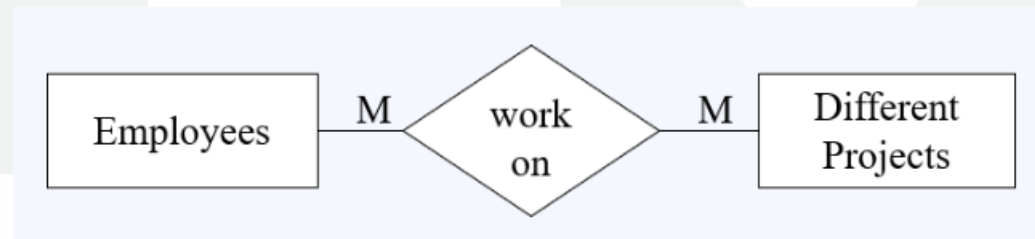


Types of Cardinality

- Many-to-One cardinality (m:1)



- Many-to-Many cardinality (m:m)





Topic

Constraints & Keys



Primary Key

There can be more than one candidate key in relation out of which one can be chosen as the primary key. For Example, STUD_NO, as well as STUD_PHONE, are candidate keys for relation STUDENT but STUD_NO can be chosen as the [primary key](#)

Example:

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	123456789
2	Rakesh	Kolkata	223365796
3	Suraj	Delhi	175468965

Foreign Key

A **foreign key** is an attribute in one table that refers to the **primary key** in another table. The table that contains the foreign key is called the **referencing table**, and the table that is referenced is called the **referenced table**.

Example: Consider the STUDENT_COURSE table

STUD_NO	TEACHER_NO	COURSE_NO
1	005	C001
2	056	C005



Super Key

The set of one or more attributes (columns) that can uniquely identify a tuple (record) is known as Super Key. It may include extra attributes that aren't essential for uniqueness but still uniquely identify the row. For Example, STUD_NO, (STUD_NO, STUD_NAME), etc.

A super key is a group of single or multiple keys that uniquely identifies rows in a table. It supports NULL values in rows.

A super key can contain extra attributes that aren't necessary for uniqueness. For example, if the "STUD_NO" column can uniquely identify a student, adding "SNAME" to it will still form a valid super key, though it's unnecessary.

Example: Consider the STUDENT table

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	123456789
2	Rakesh	Kolkata	223365796
3	Suraj	Delhi	175468965



Candidate Key

The minimal set of attributes that can uniquely identify a tuple is known as a candidate key. For Example, STUD_NO in STUDENT relation.

- A candidate key is a minimal super key, meaning it can uniquely identify a record but contains no extra attributes.
- It is a super key with no repeated data is called a candidate key.
- The minimal set of attributes that can uniquely identify a record.
- A candidate key must contain unique values, ensuring that no two rows have the same value in the candidate key's columns.
- Every table must have at least a single candidate key.
- A table can have multiple candidate keys but only one primary key.

Example: For the `STUDENT` table below, `stud_no` can be a candidate key, as it uniquely identifies each record.

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	123456789

Not Null Constraint / Check Constraint

NOT NULL Constraint:

- Ensures a column **cannot have NULL values**.
- Used when a field must always contain a value.
- Example: name VARCHAR(100) NOT NULL.

CHECK Constraint:

Limits the values that can be placed in a column.
Ensures data validity.
Example: age INT CHECK (age >= 18).



What is Relational Algebra?

The Relational Algebra consists of a set of operations that take one or two relations as input and produce a new relation as their result.

Unary Operations: They operate on one relation.

1. Select Operation
2. Project Operation

Binary Operations: They operate on more than one relations.

1. Cartesian Product Operation
2. Join Operations
3. Set Operations
4. Assignment Operation

Select Operation

- The select operation selects tuples that satisfy a given predicate.
- The lowercase Greek letter sigma (σ) is used to denote selection.
- The predicate appears as a subscript to σ .
- The argument relation is in parentheses after the σ .
- It is allowed to use $=$, $\neq$, $<$, $\leq$, $>$, $\geq$, *and* ($\wedge$), *or* ($\vee$), and *not* ($\sim$) in the selection predicate.



Examples of Select Operation

To select tuples of the *instructor* relation where the instructor is in the “Physics” department, the following statement is written:

$\sigma_{dept\ name = \text{“Physics”}}(instructor)$

Out Put

<i>ID</i>	<i>name</i>	<i>dept name</i>	<i>salary</i>
22222	Einstein	Physics	95000
33456	Gold	Physics	87000



Examples of Select Operation

To select all instructors with salary greater than \$90,000, the following statement is written:

σ salary > 90000 (*instructor*)

<i>ID</i>	<i>name</i>	<i>dept name</i>	<i>salary</i>
22222	Einstein	Physics	95000
83821	Brandt	Computer	92000

Project Operation

- The project operation is a unary operation that returns its argument relation, with certain attributes left out.
- Projection is denoted by the uppercase Greek letter pi (Π).
- We list those attributes that we wish to appear in the result as a subscript to Π .



Examples of Project Operation

To select specified attributes like *ID*, *name* and *salary* of the *instructor* relation, the following statement is written:

$\Pi ID, name, salary (instructor)$



Output

<i>ID</i>	<i>name</i>	<i>salary</i>
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	Elsaid	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000



Select Operation + Product Operation

Consider the more complicated query - “Find the names of all instructors in the Physics department.” We write:

$\Pi_{name} (\sigma_{dept\ name = \text{“Physics”}} (instructor))$



Output

<i>name</i>
Einstein
Gold

Cartesian Product Operation

- The Cartesian product operation is denoted by a cross ($\times$), allows us to combine information from any two relations.
- Consider two relations named *drinks* and *meals*.

drinks relation

orange juice
tea
coffee

meals relation

omlet
sandwich
burger

- The Cartesian product of these relations is *drinks* $\times$ *meals*.

orange juice	omlet
tea	omlet
coffee	omlet
orange juice	sandwich
tea	sandwich
coffee	sandwich
orange juice	burger
tea	burger
coffee	burger



Set Operations - Union

- Consider a query to find the set of all courses taught in the Fall 2017 semester or the Spring 2018 semester. The information is contained in the *section* relation.
- To find the set of all courses taught in the Fall 2017 semester, we write:

$\Pi_{\text{course id}} (\sigma_{\text{semester} = \text{"Fall"} \wedge \text{year} = 2017} (\text{section}))$

- To find the set of all courses taught in the Spring 2018 semester, we write:

$\Pi_{\text{course id}} (\sigma_{\text{semester} = \text{"Spring"} \wedge \text{year} = 2018} (\text{section}))$

- Now, we need the union of these two sets; that is, we need all *course ids* that appear in either or both of the two relations.

$\Pi_{\text{course id}} (\sigma_{\text{semester} = \text{"Fall"} \wedge \text{year} = 2017} (\text{section})) \cup \Pi_{\text{course id}} (\sigma_{\text{semester} = \text{"Spring"} \wedge \text{year} = 2018} (\text{section}))$

<i>course id</i>
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101



Set Operations - Intersection

- Consider a query to find the set of all courses taught in the Fall 2017 semester and the Spring 2018 semester. The information is contained in the *section* relation.
- To find the set of all courses taught in the Fall 2017 semester, we write:

$\Pi_{course\ id} (\sigma_{semester = "Fall" \wedge year = 2017} (section))$

- To find the set of all courses taught in the Spring 2018 semester, we write:

$\Pi_{course\ id} (\sigma_{semester = "Spring" \wedge year = 2018} (section))$

- Now, we need the intersection of these two sets; that is, we need all *course ids* that appear in either or both of the two relations.

$\Pi_{course\ id} (\sigma_{semester = "Fall" \wedge year = 2017} (section)) \cap \Pi_{course\ id} (\sigma_{semester = "Spring" \wedge year = 2018} (section))$

<i>course id</i>
CS-101

Set Operations - Minus

- We can find all the courses taught in the Fall 2017 semester but not in Spring 2018 semester by writing the following set-difference operation:

$\Pi_{\text{course id}} (\sigma_{\text{semester} = \text{"Fall"} \wedge \text{year} = 2017} (\text{section})) - \Pi_{\text{course id}} (\sigma_{\text{semester} = \text{"Spring"} \wedge \text{year} = 2018} (\text{section}))$

<i>course id</i>
CS-347
PHY-101



Assignment Operation

- It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.
- The assignment operation, denoted by $\leftarrow$. It is a convenient way to express complex queries.

courses fall 2017 $\leftarrow \Pi_{\text{course id}}(\sigma_{\text{semester} = \text{"Fall"} \wedge \text{year} = 2017}(\text{section}))$

courses spring 2018 $\leftarrow \Pi_{\text{course id}}(\sigma_{\text{semester} = \text{"Spring"} \wedge \text{year} = 2018}(\text{section}))$

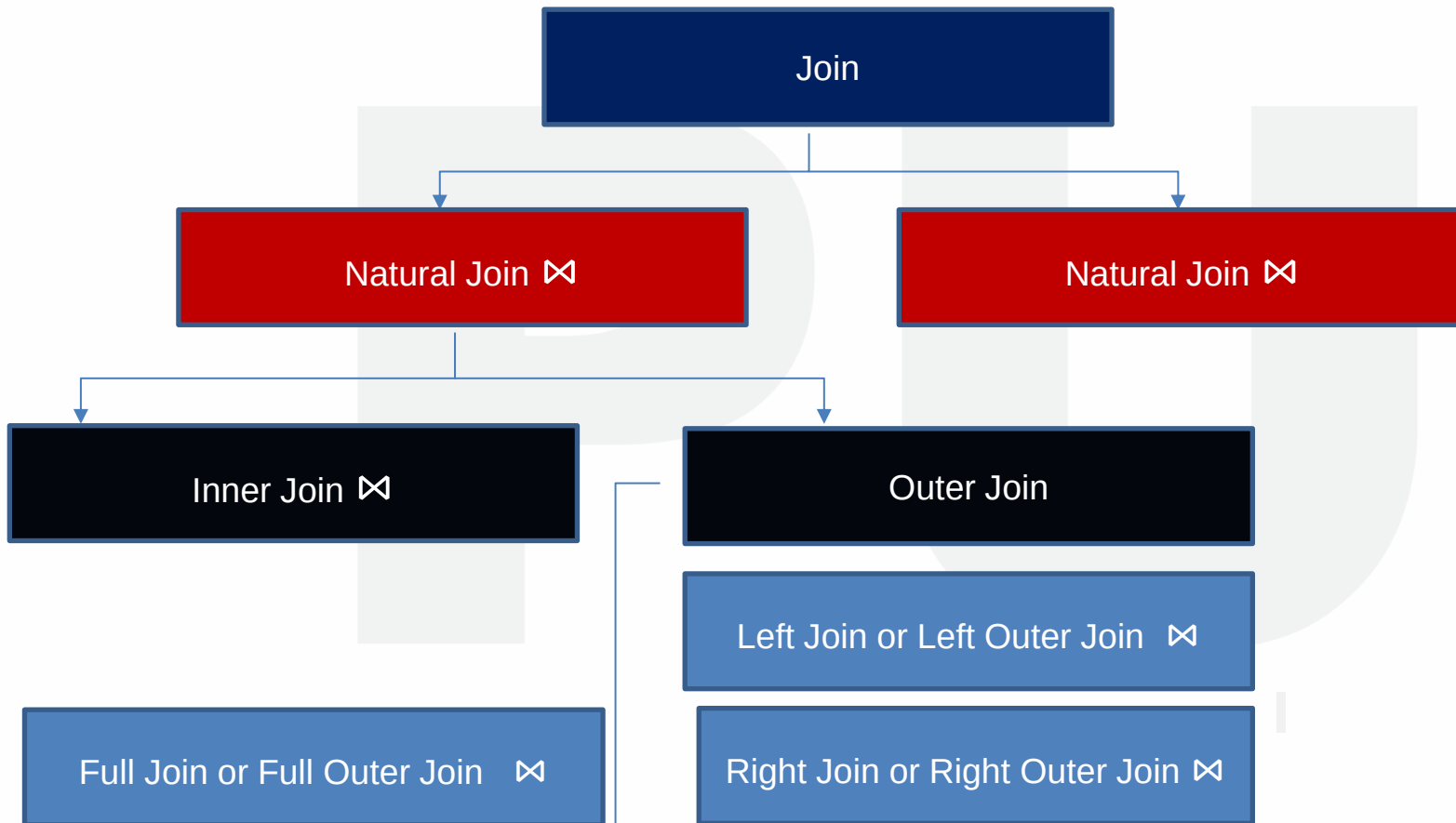
courses fall 2017 $\cap$ *courses spring 2018*

Join Operations

- The join operation allows us to combine a selection and a Cartesian product into a single operation. Join operation is denoted by “ $\bowtie$ ”.
- Consider relations $r(R)$ and $s(S)$, and let θ be a predicate on attributes in the schema $R \cup S$. The join operation is defined as follows:

$$r \bowtie_{\theta} s = \sigma_{\theta}(r \times s)$$

Types of Join Operations





ID	NAME	DEPARTMENT	SALARY
1	A	CSE	30000
2	B	CSE	35000
3	C	EE	15000
6	D	MT	50000
7	E	ME	36000
9	F	CSE	56000

INSTRUCTOR RELATION

ID	COURSE_ID	SEMESTER
1	101	1
2	203	1
6	201	2
8	304	1
9	495	1
10	320	2

TEACHES RELATION

Example of Inner Join Operation

- If we want to join *instructor* and *teaches* relations, we have to find out on the bases of which attribute we can join those relations. *ID* attribute is common and using it we can join these two relations. The following syntax can be used for inner join:

$\sigma_{\text{instructor.ID=teaches.ID}}(\text{instructor} \times \text{teaches})$

Or

$(\text{instructor}) \bowtie_{\text{instructor.ID=teaches.ID}} (\text{teaches})$



Output of Inner Join:

ID	NAME	DEPARTMENT	SALARY	COURSE_ID	SEMESTER
1	A	CSE	30000	101	1
2	B	CSE	35000	203	1
6	D	MT	50000	201	2
9	F	CSE	56000	495	1



Example of Left/Left Outer Join Operation

- The following syntax can be used for Left/Left Outer Join:

$\sigma_{\text{instructor.ID=teaches.ID}}(\text{instructor} \times \text{teaches})$

or

$(\text{instructor}) \bowtie_{\text{instructor.ID=teaches.ID}} (\text{teaches})$

Output of Left/Left Outer Join:

ID	NAME	DEPARTMENT	SALARY	COURSE_ID	SEMESTER
1	A	CSE	30000	101	1
2	B	CSE	35000	203	1
3	C	EE	15000	NULL	NULL
6	D	MT	50000	201	2
7	E	ME	36000	NULL	NULL
9	F	CSE	56000	495	1

Example of Right/Right Outer Join Operation

- The following syntax can be used for Right/Right Outer Join:

$\sigma_{\text{instructor.ID=teaches.ID}}(\text{instructor} \times \text{teaches})$

Or

$(\text{instructor}) \bowtie_{\text{instructor.ID=teaches.ID}} (\text{teaches})$

Output of Right/Right Outer Join:

ID	NAME	DEPARTMENT	SALARY	COURSE_ID	SEMESTER
1	A	CSE	30000	101	1
2	B	CSE	35000	203	1
6	D	MT	50000	201	2
8	NULL	NULL	NULL	304	1
9	F	CSE	56000	495	1
10	NULL	NULL	NULL	320	2



Example of Full/Full Outer Join Operation

- The following syntax can be used for Full/Full Outer Join:

$\sigma_{\text{instructor.ID}=\text{teaches.ID}}(\text{instructor} \times \text{teaches})$

Or

$(\text{instructor}) \bowtie_{\text{instructor.ID}=\text{teaches.ID}} (\text{teaches})$

Output of Full/Full Outer Join:

ID	NAME	DEPARTMENT	SALARY	COURSE_ID	SEMESTER
1	A	CSE	50000	101	1
2	B	CSE	10000	201	1
3	C	EE	15000	303	2
6	D	MT	20000	507	1
7	E	ME	25000	NULL	NULL
8	NULL	NULL	35000	706	1
9	F	CSE	NULL	907	2
10	NULL	NULL	NULL	NULL	NULL